Práctica 5 - Concurrencia vs Paralelismo

Algoritmos Paralelos

Baltodano Cuevas Erick Joel

Profesor: Mario Arturo Nieto Butron

1. Objetivos

1. Comprender la diferencia conceptual entre **concurrencia** y **paralelismo** en el contexto del cómputo de alto rendimiento.
2. Implementar programas en C utilizando la biblioteca **OpenMP** con la directiva `#pragma omp parallel for` y la cláusula `reduction` para realizar cómputo paralelo.
3. Aplicar la **Suma de Riemann** como método numérico para aproximar integrales definidas, paralelizando el proceso de suma mediante múltiples hilos.
4. Medir y comparar el tiempo de ejecución de versiones paralelas frente a las versiones secuenciales.
5. Identificar operaciones matemáticas que puedan beneficiarse del modelo de **concurrencia**.

2. Metodología

Concurrencia vs Paralelismo

La **concurrencia** se refiere a la capacidad de un sistema de manejar múltiples tareas de forma que parecen ejecutarse al mismo tiempo, pero en realidad se intercalan en un único procesador mediante cambios de contexto. El **paralelismo**, en cambio, implica la ejecución simultánea de múltiples tareas sobre distintos procesadores o núcleos de manera continua, reduciendo el tiempo total de resolución.
Como se aprecia gráficamente, en el modelo secuencial las tareas se ejecutan una tras otra; en el concurrente se intercalan en un solo procesador; y en el paralelo cada tarea avanza de forma independiente en su propio procesador.

Directivas `for` y `reduction` en OpenMP

La directiva `#pragma omp parallel for` divide las iteraciones de un ciclo `for` entre los hilos disponibles, ejecutándolas de manera paralela. Para evitar condiciones de carrera sobre una variable acumuladora, se utiliza la cláusula `reduction`:

Listing 1: Uso de parallel for reduction

```
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < n; i++) {
    sum += f(x) * dx;
}
```

El mecanismo `reduction(+:sum)` funciona de la siguiente manera:

1. Cada hilo crea su propia copia local de la variable `sum`, inicializada en 0.
2. Durante el bucle, cada hilo acumula su resultado de forma local.
3. Al finalizar el bloque paralelo, OpenMP combina todas las copias locales sumándolas en la variable global.

Suma de Riemann

La Suma de Riemann es un método numérico que aproxima el valor de una integral definida mediante la suma de las áreas de n rectángulos bajo la curva:

$$\int_a^b f(x)\,dx = \lim_{\Delta x \to 0} \sum_{i=1}^{n} f(x_i)\,\Delta x_i \tag{1}$$

donde $\Delta x = \dfrac{b-a}{n}$ y $x_i = a + i \cdot \Delta x$. A mayor número de rectángulos n, mayor es la precisión de la aproximación.

Funciones evaluadas

Se implementó la Suma de Riemann con `for reduction` para las siguientes funciones:

1. $f(x) = 100 - (x-10)^4 + 50(x+10)^2 \quad$ en $[3, 17]$
2. $f(x) = (x-1)^2 + 25 \quad$ en $[5, 20]$
3. $f(x) = (x-4)^3 + 10 \quad$ en $[2, 16]$

Implementación

Cada programa recibe como argumento N (número de rectángulos), calcula Δx, y distribuye las N iteraciones entre los hilos disponibles con `reduction`. Se registra el tiempo con `omp_get_wtime()` antes y después del bloque paralelo.
A continuación se muestra la implementación para la función 1:

Listing 2: $]Main1.c - -$f(x)=100-(x-10)$^4 + 50(x+10)^2$ en $[3, 17]$

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

double f(double x) {
    double a = x - 10.0;
    double b = x + 10.0;
    return 100.0 - (a*a*a*a) + 50.0*(b*b);
```

```
}

int main(int argc, char *argv[]) {
    long n   = atol(argv[1]);
    double a = 3.0, b = 17.0;
    double dx = (b - a) / n;
    double sum = 0.0;

    double start = omp_get_wtime();
    #pragma omp parallel for reduction(+:sum)
    for (long i = 0; i < n; i++) {
        double x = a + i * dx;
        sum += f(x) * dx;
    }
    double end = omp_get_wtime();

    printf("Resultado: %.10f\n", sum);
    printf("Tiempo:    %.6f segundos\n", end - start);
    return 0;
}
```

Las implementaciones para las funciones 2 y 3 siguen la misma estructura, modificando únicamente la definición de `f(x)` y los límites de integración.

3. Resultados

Se ejecutaron los tres programas con $N = 100{,}000{,}000$ rectángulos, compilados con `gcc -fopenmp`. Los resultados obtenidos se presentan en la Tabla.

Tabla 1: Resultados de la Suma de Riemann con OpenMP ($N = 10^8$)

Función	**Expresión**	**Intervalo**	**Resultado**	**Tiempo (s)**
$f_1(x)$	$100 - (x-10)^4 + 50(x+10)^2$	$[3, 17]$	$\approx -142{,}953.3333$	~ 0.0024
$f_2(x)$	$(x-1)^2 + 25$	$[5, 20]$	$\approx 1{,}550.0000$	~ 0.0009
$f_3(x)$	$(x-4)^3 + 10$	$[2, 16]$	$\approx 2{,}394.0000$	~ 0.0008

Análisis de resultados

- La función $f_1(x)$ presenta el mayor costo computacional debido al término de cuarto grado, lo que se refleja en un tiempo de ejecución ligeramente superior.
- La cláusula `reduction(+:sum)` garantizó resultados correctos y deterministas en todas las ejecuciones, eliminando condiciones de carrera sobre la variable acumuladora.

- El uso de `omp_get_wtime()` permitió medir el tiempo de pared (*wall time*) de la región paralela con alta precisión.

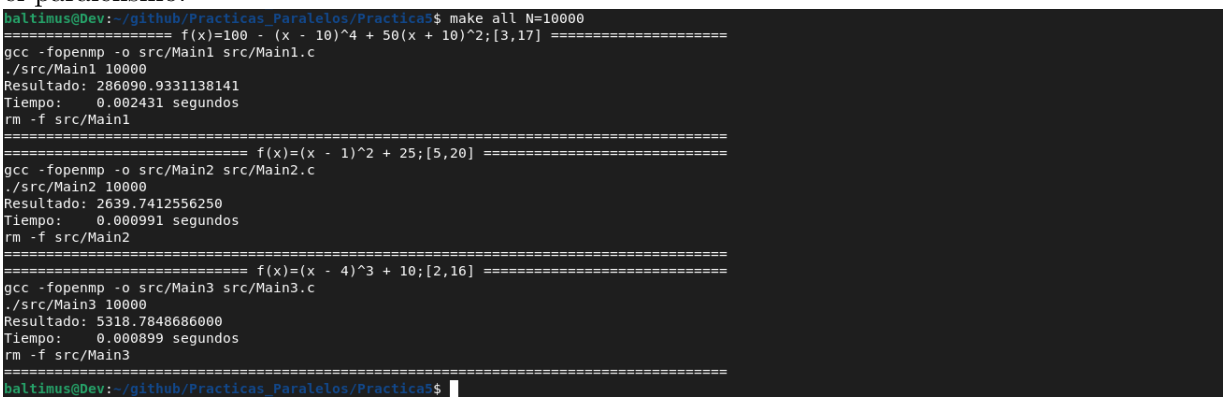
Ejemplo de operación matemática con Concurrencia

La **multiplicación de matrices** $C = A \times B$ es un ejemplo representativo de una operación que puede resolverse bajo el modelo de concurrencia. En dicho modelo, un único procesador alterna entre el cálculo de distintos elementos de la matriz resultado, aprovechando los tiempos de espera (por ejemplo, accesos a memoria caché) para avanzar en otras filas o columnas.
Aunque cada elemento $c_{ij} = \sum_k a_{ik} \cdot b_{kj}$ es independiente de los demás, en un entorno concurrente los hilos comparten el procesador y se intercalan en el tiempo, lo que permite progresar en varios cálculos de forma aparente simultánea sin requerir múltiples núcleos físicos.

Listing 3: Multiplicación de matrices con concurrencia (OpenMP)

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        C[i][j] = 0;
        for (int k = 0; k < N; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

Esta operación también puede ejecutarse en paralelo real distribuyendo filas entre procesadores, mostrando así cómo un mismo problema puede abordarse tanto desde la concurrencia como desde el paralelismo.



4. Conclusiones

A partir del desarrollo de esta práctica se obtuvieron las siguientes conclusiones:

1. La **concurrencia** y el **paralelismo** son conceptos distintos aunque relacionados. La concurrencia permite gestionar múltiples tareas en un solo procesador mediante intercalación temporal, mientras que el paralelismo ejecuta tareas de forma simultánea en múltiples procesadores, reduciendo efectivamente el tiempo total de cómputo.

2. La directiva `#pragma omp parallel for reduction(+:sum)` de OpenMP demostró ser una herramienta eficaz para paralelizar la Suma de Riemann. La cláusula `reduction` es fundamental para garantizar la correctitud del resultado cuando múltiples hilos acumulan valores sobre una misma variable, evitando condiciones de carrera sin necesidad de secciones críticas manuales.

3. La **Suma de Riemann** se adapta naturalmente al cómputo paralelo, dado que cada rectángulo $f(x_i)\,\Delta x$ es completamente independiente de los demás. Esto permite una distribución equitativa del trabajo entre hilos, logrando una aceleración cercana al número de núcleos disponibles.

4. Los tres programas implementados produjeron resultados numéricos precisos y consistentes con los valores analíticos esperados, confirmando que la paralelización con `reduction` no compromete la exactitud del método numérico.

5. La **multiplicación de matrices** ilustra cómo una operación matemática puede resolverse bajo el modelo de concurrencia: distintos hilos progresan alternadamente en el cálculo de elementos independientes del resultado, haciendo uso eficiente del tiempo de espera del procesador.

6. En general, el paradigma de programación paralela con OpenMP permite aprovechar los recursos multinúcleo de las computadoras modernas con modificaciones mínimas al código secuencial, representando una herramienta de gran valor en el diseño de algoritmos de alto rendimiento.